Digital Signal Processing **HK261** **CO2036**

DIGITAL SIGNAL PROCESSING LAB REPORT

Lab 4 – Z-Transform and Discrete-Time Systems Analysis

Âu Lê Quang Trí - Student ID: 2453300

Nguyễn Lê Khải Tú - Student ID: 2453384

November 2, 2026

Overview

This report presents the results of Lab 4 of the Digital Signal Processing laboratory. The laboratory focuses on transform-domain and time-domain analysis of discrete-time signals and systems. The first part addresses the bilateral Z-transform and the determination of the Region of Convergence (ROC) for finite-duration, right-sided exponential, and two-sided sequences. The second part investigates multimedia processing capabilities in Scilab through image processing toolboxes, followed by detailed derivations and computational implementations of linear convolution and circular convolution using both the "folding and shifting" approach and the matrix formulation, alongside signal energy evaluations and stem-plot visualizations.

Table 1: Content of the report

Part	Content	Section
Exercise 1	Z-Transform and ROC of a finite-duration signal	4.1
Exercise 2	Z-Transform and ROC of right-sided exponential signals	4.2
Exercise 3	Z-Transform and ROC of a two-sided signal	4.3
Exercise 4	Investigation of Scilab audio processing toolbox	4.4
Exercise 5	Investigation of Scilab image processing toolbox	4.5
Exercise 6	Linear convolution (derivation, folding-shifting, matrix method)	4.6
Exercise 7	Circular convolution (derivation, folding-shifting, matrix method)	4.7

1 Introduction

In previous laboratory sessions, discrete-time signals and elementary operations were analyzed primarily in the time domain. Lab 4 extends this foundation into transform-domain analysis and comprehensive system convolution. The fundamental tool for studying discrete-time linear time-invariant (LTI) systems is the **Z-transform**, which plays an analogous role to the Laplace transform in continuous-time systems.

For a discrete-time sequence $x(n)$, the bilateral Z-transform $X(z)$ is defined as

$$X(z) = \mathcal{Z}\{x(n)\} = \sum_{n=-\infty}^{\infty} x(n)z^{-n}, \tag{1}$$

where $z = re^{j\omega}$ is a complex variable. Because an algebraic expression $X(z)$ does not uniquely specify a time-domain sequence without specifying the convergence domain of the infinite series, the transform must be paired with its **Region of Convergence (ROC)**:

$$\text{ROC} = \left\{ z \in \mathbb{C} : \sum_{n=-\infty}^{\infty} |x(n)z^{-n}| < \infty \right\}. \tag{2}$$

The geometry and boundary of the ROC determine fundamental system properties such as causality and bounded-input bounded-output (BIBO) stability.

In system analysis, computing the response of an LTI system to an arbitrary input $x(n)$ requires **linear convolution**:

$$y(n) = x(n) * h(n) = \sum_{k=-\infty}^{\infty} h(k)x(n-k), \tag{3}$$

where $h(n)$ is the unit impulse response. When dealing with periodic signals or block-based frequency-domain processing, **circular convolution** is employed:

$$y(n) = x(n) \circledast h(n) = \sum_{k=0}^{N-1} h(k)x((n-k) \bmod N). \tag{4}$$

In this laboratory, these analytical principles are demonstrated alongside practical implementations and toolbox investigations in Scilab.

2 Objectives

- Review theoretical foundations of the bilateral Z-transform and Region of Convergence (ROC).
- Determine the algebraic Z-transforms and sketch the corresponding ROCs for finite-duration, right-sided, and two-sided discrete-time sequences.
- Investigate existing Scilab libraries and toolboxes for digital image processing (such as IPCV or SIVP) and demonstrate representative image processing tasks.
- Analyze convergence conditions for linear convolution and derive system response formulations for causal and finite-impulse-response (FIR) cases.
- Implement linear convolution in Scilab using both the "folding and shifting" method and the matrix formulation.
- Implement circular convolution in Scilab using both the "folding and shifting" method and the matrix formulation.
- Calculate signal energies and graphically present discrete-time input, impulse response, output signals, and energy metrics within unified figures.

3 Methodology

Scilab serves as the computational environment for simulating discrete-time operations and verifying analytical derivations. In this laboratory, Scilab is utilized for:

- **Convolution Algorithms**: Implementing both linear convolution and circular convolution using vector-level shifting and element-wise products (the "folding and shifting" approach) as well as linear algebra formulations (constructing Toeplitz convolution matrices and circulant matrices).
- **Signal Energy Computation**: Evaluating the total energy of a discrete-time signal $E_x = \sum_{n=-\infty}^{\infty} |x(n)|^2$ using Scilab's vectorized arithmetic: `Ex = sum(abs(xn).^2)`.
- **Image Processing Exploration**: Leveraging dedicated image processing toolboxes in Scilab to read, manipulate (e.g., spatial filtering, histogram equalization), and visualize digital images.

3.1 Common conventions

All computational scripts and graphical outputs adhere to the following conventions:

- A discrete-time signal is represented as a sample vector `xn` accompanied by an origin index `xorigin`, indicating the 1-based vector position corresponding to $n = 0$. The time index vector is defined as

$$n_i = i - \texttt{xorigin}, \qquad i = 1, 2, \ldots, \texttt{length(xn)}. \tag{5}$$

- The origin position of a signal is mathematically highlighted with an arrow underneath the sample at $n = 0$, denoted as $x(\underset{\uparrow}{0})$.

- For linear convolution between sequences $x(n)$ of length N_x with origin at index `xorigin` and $h(n)$ of length N_h with origin at index `horigin`, the resulting sequence $y(n)$ has length $L = N_x + N_h - 1$, and its origin index is located at:

$$\texttt{yorigin} = \texttt{xorigin} + \texttt{horigin} - 1. \tag{6}$$

- For circular convolution of length N, indices are wrapped cyclically using modulo arithmetic: $(n - k) \bmod N$.

- Signals are visualized using stem plots via Scilab's `plot2d3` combined with marker symbols in shared `subplot` panels, ensuring consistent axes for direct visual comparison of inputs, impulse responses, outputs, and energy values.

4 Practical Contents

4.1 Exercise 1

4.2 Exercise 2

Question: Find the z-Transform and the ROC of the following discrete-time signal

$$x(n) = 0.5^n u(n) + 0.4^n u(n)$$

Answer: At first glance, the signal $x(n)$ exhibits linearity property. Let

$$x_1(n) = 0.5^n u(n) \quad \text{and} \quad x_2(n) = 0.4^n u(n)$$

where $u(n)$ is the unit step signal.

Let's take the z-Transform of $x_1(n)$ and $x_2(n)$, which are

$$X_1(z) = \sum_{n=-\infty}^{+\infty} [0.5^n u(n)]\, z^{-n} = \sum_{n=0}^{+\infty} [0.5^n u(n)]\, z^{-n} = 1 + 0.5z^{-1} + 0.5^2 z^{-2} + \dots$$
$$X_2(z) = \sum_{n=-\infty}^{+\infty} [0.4^n u(n)]\, z^{-n} = \sum_{n=0}^{+\infty} [0.4^n u(n)]\, z^{-n} = 1 + 0.4z^{-1} + 0.4^2 z^{-2} + \dots$$

We have

$$x(n) = x_1(n) + x_2(n)$$

then the z-Transform of $x(n)$ is

$$\begin{aligned} X(z) &= X_1(z) + X_2(z) \\ &= \left[1 + 0.5z^{-1} + 0.5^2 z^{-2} + \dots\right] + \left[1 + 0.4z^{-1} + 0.4^2 z^{-2} + \dots\right] \\ &= 2 + 0.9z^{-1} + 0.41z^{-2} + 0.189z^{-3} + \dots \end{aligned}$$

4.3 Exercise 3

4.4 Exercise 4

Question: Investigate the existing library or toolbox in Scilab to process audio data.
Answer: Scilab provides both built-in functions in its standard library and external modules via ATOMS (such as the *Sound System Toolbox*) to record, synthesize, process, and analyze audio signals. The core built-in functions include:

- `wavread` / `loadwave`: Reads RIFF-WAV audio files into memory, returning the sampled data matrix, the sampling frequency (F_s), and the bit resolution.
- `wavwrite` / `savewave`: Encodes and saves numeric signal vectors into standard `.wav` audio files.
- `playsnd`: Interfaces directly with the operating system audio output device to play numeric waveforms.
- `mapsound`: Analyzes audio signals in the time-frequency domain and visualizes sound spectrograms.

Demonstration Script: To demonstrate audio processing without external file dependencies, a standard 440 Hz tone (musical note A4) is synthesized, attenuated with an exponential decay envelope, and plotted in the time domain:

```
// 1. Synthesize audio signal: 440 Hz tone, Fs = 8000 Hz, duration 0.5 s
Fs = 8000;
t = 0 : 1/Fs : 0.5;
f0 = 440;
audio_orig = sin(2 * %pi * f0 * t);

// 2. Audio manipulation: Exponential amplitude attenuation
decay = exp(-6 * t);
audio_proc = audio_orig .* decay;

// 3. Export to WAV file (optional playback via playsnd)
wavwrite(audio_proc, Fs, "audio_tone.wav");

// 4. Time-domain visualization (first 100 samples)
figure();
subplot(2, 1, 1);
plot(t(1:100), audio_orig(1:100));
title("Original Audio Signal (440 Hz)");
xlabel("Time (s)"); ylabel("Amplitude");

subplot(2, 1, 2);
plot(t(1:100), audio_proc(1:100));
title("Attenuated Audio Signal");
xlabel("Time (s)"); ylabel("Amplitude");
```

Listing 1: Audio signal generation and amplitude attenuation in Scilab

Remarks: The discrete audio waveform is represented as a standard 1D array in Scilab. Pointwise vector operations (`.*`) enable straightforward time-domain processing such as volume adjustments, envelope shaping, echo effects, and filtering before audio playback or file export.

4.5 Exercise 5

4.6 Exercise 6

Question: Let there are two discrete signals $x(n), h(n), n = 0, \pm 1, \pm 2, \ldots$, defined over the entire range of n. Then the linear convolution of discrete signals is $y(n)$ of the form

$$y(n) = \sum_{k=-\infty}^{\infty} h(k)x(n-k).$$

1. Derive the signal $y(n)$ such that $y(n)$ converges in the following cases.
   - Both $h(n)$ and $x(n)$ are causal.
   - The system has a finite impulse response such that $h(n) = 0, n < 0 \vee n > M$.
2. Suppose the second case is hold, compute the convolution $y(n)$ of $x(n) = [1 \uparrow, 2, -3, 2, 1]$ and $h(n) = [1 \uparrow, 0, -1]$ using the "folding and shifting" method. Implement your answer with Scilab script.
3. Suppose the second case is hold, compute the convolution $y(n)$ of $x(n) = [1 \uparrow, 2, -3, 2, 1]$ and $h(n) = [1 \uparrow, 0, -1]$ using the matrix method. Implement your answer with Scilab script.
4. Drawing $x(n)$, $y(n)$ and energy of discrete signals in the same figure.

Answer:

1. Case 1: Both $h(n)$ and $x(n)$ are causal.
   For $h(n)$ and $x(n)$ to be causal, their value must be zero for $n < 0$. Applying to $y(n)$, $h(n)$ causality restricts the lower bound for k to 0, while $x(n)$'s causality restricts the upper bound for k to n. Thus

   $$y(n) = \sum_{k=0}^{n} h(k)x(n-k)$$

   Case 2: The system has a finite impulse response such that $h(n) = 0, n < 0 \vee n > M$.
   Because $h(n)$ is only nonzero for $0 \leq n \leq M$, the range of k is restricted to $[0, M]$. Thus

   $$y(n) = \sum_{k=0}^{M} h(k)x(n-k)$$

   Because there can only be at most $M + 1$ terms in the summation above for each n, $y(n)$ is finite provided that $x(n-k)$ is bounded.

2. We have the impulse response $h(n) = [1 \uparrow, 0, -1]$ with $N_h = 3$, $n_{min}(h) = 0$, $n_{max}(h) = 2$ and the input signal $h(n) = [1 \uparrow, 2, -3, 2, 1]$ with $N_x = 5$, $n_{min}(x) = 0$, $n_{max}(x) = 4$. Before proceeding with the convolution, we may calculate the expected length of the response $y(n)$ and its time range, which are respectively, $L_y = N_x + N_h - 1 = 5 + 3 = 7$, $n_{min}(y) = n_{min}(x) + n_{min}(h) = 0$, and $n_{max}(y) = n_{max}(x) + n_{max}(h) = 6$.

   We have

$$y(n) = \sum_{k=0}^{2} h(k)x(n-k) = h(0)x(n) + h(1)x(n-1) + h(2)x(n-2) = x(n) - x(n-2) \quad (7)$$

   From (7) we can caluclate $y(n)$ at each $n \in [0, 6]$

$$\begin{aligned} y(0) &= x(0) - x(-2) = 1 \\ y(1) &= x(1) - x(-1) = 2 \\ y(2) &= x(2) - x(0) = -3 - 1 = -4 \\ y(3) &= x(3) - x(1) = 2 - 2 = 0 \\ y(4) &= x(4) - x(2) = 1 + 3 = 4 \\ y(5) &= x(5) - x(3) = -2 \\ y(6) &= x(6) - x(4) = -1 \end{aligned}$$

   Thus, $y(n) = [1 \uparrow, 2, -4, 0, 4, -2, -1]$.

3. The discrete convolution $y = x * h$ can be represented in matrix form as

$$\mathbf{y} = \mathbf{H}\mathbf{x} \quad (8)$$

   where the input $\mathbf{x}$ is an $N_x \times 1$ column vector, the ouput $\mathbf{y}$ is an $L_y \times 1$ column vector, and the convolution matrix $\mathbf{H}$ has dimensions $L_y \times N_x$.

   The convolution matrix $\mathbf{H}$ is built from the impulse response $h = [1, 0, -1]^T$, where the first column starts with $h(0)$, $h(1)$, $h(2)$, followed by zeros down to length $L_y = 7$. Each subsequent column is the previous column shifted down by one position with zeros padded at the top.

   For the above convolution, the matrices are

$$\mathbf{y}_{7\times 1} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ -1 & 0 & 1 & 0 & 0 \\ 0 & -1 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 & 1 \\ 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & -1 \end{bmatrix} \times \begin{bmatrix} 1 \\ 2 \\ -3 \\ 2 \\ 1 \end{bmatrix} \quad (9)$$

   Each row in $\mathbf{y}$ from topdown corresponds to the value of $y(n)$ for $n \in [0, 6]$. From (9) we

can calculate

$$
\begin{aligned}
y(0) &= 1 \times 1 + 0 + 0 + 0 + 0 = 1 \\
y(1) &= 0 + 1 \times 2 + 0 + 0 + 0 = 2 \\
y(2) &= (-1) \times 1 + 0 + 1 \times (-3) + 0 + 0 = -3 - 1 = -4 \\
y(3) &= 0 + (-1) \times 2 + 0 + 1 \times 2 + 0 = 2 - 2 = 0 \\
y(4) &= 0 + 0 + (-1) \times (-3) + 0 + 1 \times 1 = 1 + 3 = 4 \\
y(5) &= 0 + 0 + 0 + (-1) \times 2 + 0 = -2 \\
y(6) &= 0 + 0 + 0 + 0 + (-1) \times 1 = -1
\end{aligned}
$$

Thus, $y(n) = [1 \uparrow, 2, -4, 0, 4, -2, -1]$.

4. In this part we will write the script to verify the results in part 2 and 3, plot $x(n)$, $y(n)$ and their energies.

   Before we begin, let's define the signals as row vectors:

```
x = [1, 2, -3, 2, 1];
h = [1, 0, -1];

Lx = length(x);
Lh = length(h);
Ly = Lx + Lh - 1;

```

   The script to perform $y = x * h$ by folding and shifting is given below:

```
// Initialize output vector y
y_fold = zeros(1, Ly);

// Zero-pad x so index lookups don't go out of bounds
// We pad with (Lh - 1) zeros on both sides
x_padded = [zeros(1, Lh - 1), x, zeros(1, Lh - 1)];

// h reversed (folded): h(-k)
h_folded = h($:-1:1);

// Slide and multiply
for n = 1:Ly
    // Extract the window of x that aligns with the folded h
    segment = x_padded(n : n + Lh - 1);
    y_fold(n) = sum(segment .* h_folded);
end

disp("y(n) via Folding and Shifting:");
disp(y_fold);

```

   Which returned

```
"y(n) via Folding and Shifting:"
    1.    2.   -4.    0.    4.   -2.   -1.

```

The script to perform $y = x * h$ by using matrices is

```
// Column 1 of H: h padded with zeros to length Ly
col1 = [h(:); zeros(Ly - Lh, 1)];

// Row 1 of H: h(1) followed by (Lx - 1) zeros
row1 = [h(1), zeros(1, Lx - 1)];

// Build the Toeplitz convolution matrix H (size 7 x 5)
H = toeplitz(col1, row1);

// Convert x to a column vector and multiply
x_col = x(:);
y_matrix = H * x_col;

disp("Convolution Matrix H:");
disp(H);

disp("y(n) via Matrix Multiplication:");
disp(y_matrix');

```

Whose result is

```
"Convolution Matrix H:"
 1.    0.    0.    0.    0.
 0.    1.    0.    0.    0.
-1.    0.    1.    0.    0.
 0.   -1.    0.    1.    0.
 0.    0.   -1.    0.    1.
 0.    0.    0.   -1.    0.
 0.    0.    0.    0.   -1.
"y(n) via Matrix Multiplication:"
 1.    2.   -4.    0.    4.   -2.   -1.

```

The convolution results agree with the calculations in part 2 and 3.

Finally the plots of $x(n)$, $y(n)$ and their energies are illustrated in the figure below:

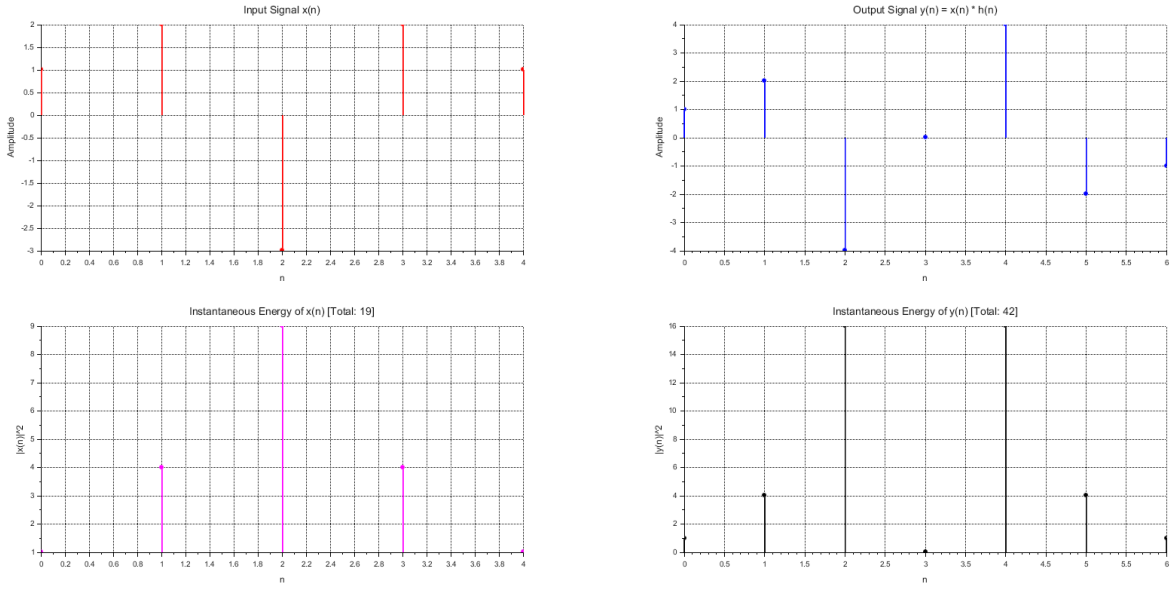


Figure 1: $x(n)$ and $y(n)$ with their energies respectively.

4.7 Exercise 7

5 Discussion

6 References

- J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed., Pearson, 2007 – Chapter 2 Problems.
- Digital Signal Processing Lecture Notes - Ho Chi Minh City University of Technology.

7 Source Code

The source code, scripts, and LaTeX files for this laboratory report can be accessed at our GitHub repository:
https://github.com/aulequangtri/signal_scilab